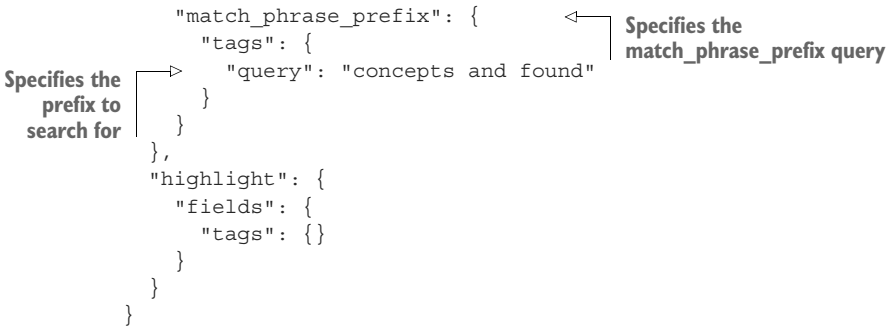


This query fetches all books with tags matching "found". It matches "foundational" in our books index.

Similar to a `match_phrase` query, word order is also important in a `match_phrase_prefix` query. And again, `slop` comes to the rescue. For example, to retrieve books with the phrase *concepts and foundations* in the `tags` field, we can drop a word by adding the `slop` keyword.

Listing 10.13 Using a `match_phrase_prefix` query with `slop`

```
# Match phrase prefix
GET books/_search
{
  "query": {
    "match_phrase_prefix": {
      "tags": {
        "query": "concepts found",
        "slop":1
      }
    }
  }
}
```

The phrase omits one word ("and") and has a prefix ("found").

Sets slop to 1 because one word is dropped from the phrase

Setting the `slop` keyword to `1` queries books with the tags `concepts` and `found*`, but it ignores the word *and*. The query should return the book *Kotlin Programming* as the result because the query matches the phrase "Kotlin concepts and foundational APIs" in the `tags` field.

So far, we've queried search criteria across a single field. However, let's say we want to find the words "Software Development" across the `title`, `synopsis`, and `tags` fields. That's what happens when we use a `multi_match` query, discussed in the next section.

10.8 The multi_match query

As the name suggests, a multi-match (`multi_match`) query searches across multiple fields. For example, to search for the word "Java" across the `title`, `synopsis`, and `tags` fields, a `multi_match` query is the answer. The following listing shows a query that searches for "Java" across these three fields.